02 Normalization, feature selection and clustering

This notebook performs: - Library-size log normalization, - selection of highly variable genes (HVGs) with batch blocking, - HVG principal components, and - graph-based clustering

Data are not integrated or batch-corrected. Sequencing batch is used only as a blocking factor when modelling gene variance.

```
suppressPackageStartupMessages({
  library(SingleCellExperiment)
  library(scuttle)
  library(scran)
  library(scater)
  library(igraph)
})
source(file.path("..", "scripts", "helpers.R"))
paths <- load_paths()
```

Parameters

```
n_hvg          <- 2000
hvg_pca_dims   <- 13
block_field    <- "Seq_batch"
mito_pattern   <- "^MT-"
ribo_pattern   <- "^RP[SL]"
snn_k          <- 20
leiden_res     <- c(1.0, 0.8)
leiden_seed    <- 1

# Sex-chromosome genes are taken from Ensembl BioMart (release 116 archive)
biomart_host    <- "https://jun2026.archive.ensembl.org"
biomart_dataset <- "hsapiens_gene_ensembl"
sex_chromosomes <- c("X", "Y", "chrX", "chrY", "CHR_X", "CHR_Y", "chr_X", "chr_Y")
```

```
sce <- readRDS(file.path(paths$work_dir, "01_qc_filtered.rds"))
dim(sce)
```

```
## [1] 19189  1050
```

Normalization

`logcounts = log2(counts / size factor + 1)`, using library-size factors (`size.factors = NULL`).

```
sce <- scuttle::logNormCounts(sce, transform = "log", size.factors = NULL,
                              name = "logcounts")
summary(sizeFactors(sce))
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
##  0.04127  0.24865  0.49228  1.00000  1.19883 13.33160
```

Highly variable genes

Gene variance is modelled with sequencing batch as a blocking factor. Mitochondrial, ribosomal and sex-chromosome genes are excluded from the HVG selection

```
query <- paste0(
  '<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE Query>',
  '<Query virtualSchemaName="default" formatter="TSV" header="1" uniqueRows="1" ',
  'count="" datasetConfigVersion="0.6">',
  '<Dataset name="', biomart_dataset, '" interface="default">',
  '<Attribute name="external_gene_name"/><Attribute name="chromosome_name"/>',
  '</Dataset></Query>')
bm <- utils::read.delim(
  paste0(biomart_host, "/biomart/martservice?query=",
         utils::URLencode(query, reserved = TRUE)),
  header = TRUE, stringsAsFactors = FALSE, check.names = FALSE)
names(bm) <- c("external_gene_name", "chromosome_name")
sexchr_genes <- unique(bm$external_gene_name[bm$chromosome_name %in% sex_chromosomes])

is_sex  <- as.character(rowData(sce)$symbol) %in% sexchr_genes
is_mito <- grepl(mito_pattern, rownames(sce))
is_ribo <- grepl(ribo_pattern, rownames(sce))
c(sex = sum(is_sex), mito = sum(is_mito), ribo = sum(is_ribo))
```

```
##  sex mito ribo
##  790   13   98
```

```
save_tsv(data.frame(gene_id = rownames(sce), is_sex = is_sex),
         file.path(paths$work_dir, "02_sex_chromosome_flags.tsv"))
```

```
dec <- scran::modelGeneVar(sce, assay.type = "logcounts",
                           block = as.character(sce[[block_field]]))
dec_df <- as.data.frame(dec, stringsAsFactors = FALSE, check.names = FALSE)
```

```
## Warning in .local(x, row.names, optional, ...): arguments in '...' ignored
```

```
stopifnot(identical(rownames(dec_df), rownames(sce)))

eligible <- !(is_mito | is_ribo | is_sex) & is.finite(dec_df$bio) &
 ↪  is.finite(dec_df$total)
hvgs <- as.character(scran::getTopHVGs(dec_df[eligible, , drop = FALSE], n = n_hvg))
stopifnot(length(hvgs) == n_hvg)
```

Dimensionality reduction

PCA is performed on the scaled logcounts of the 2,000 HVGs with `calculatePCA` using the default solver (randomized IRLBA).

```
set.seed(1)
hvg_pca <- scater::calculatePCA(sce, subset_row = hvgs, ncomponents = hvg_pca_dims,
                                exprs_values = "logcounts", scale = TRUE)
hvg_pca <- as.matrix(hvg_pca)[colnames(sce), , drop = FALSE]
colnames(hvg_pca) <- paste0("PC", seq_len(hvg_pca_dims))
reducedDim(sce, "hvg_pca") <- hvg_pca
```

Clustering

Shared-nearest-neighbour graph is constructed on the HVG PCs, followed by Leiden clustering.

```
graph <- scran::buildSNNGraph(t(hvg_pca), k = snn_k)
for (r in leiden_res) {
  set.seed(leiden_seed)
  field <- paste0("cluster_hvg_res", sub("\\.0$", "", format(r)))
  colData(sce)[[field]] <- as.character(
    igraph::cluster_leiden(graph, resolution_parameter = r)$membership)
}
```

```
## Warning: The `resolution_parameter` argument of `cluster_leiden()` is deprecated as of
## igraph 2.1.0.
## i Please use the `resolution` argument instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
table(sce$cluster_hvg_res1)
```

```
##
##   1  10  11   2   3   4   5   6   7   8   9
## 188  79 153  41  56 170 151 130  29  22  31
```

Verify against the deposited object

In the deposited object, `cluster_hvg_res1` is stored as `final_cluster`.

```
ann <- readRDS(file.path(paths$zenodo_root, "RDS", "filtered_annotated_object.rds"))
stopifnot(
  isTRUE(all.equal(assay(sce, "logcounts"), assay(ann, "logcounts"), check.attributes =
   ↪  FALSE)),
  max(abs(abs(reducedDim(sce, "hvg_pca")) - abs(reducedDim(ann, "hvg_pca")))) < 1e-2,
  identical(sce$cluster_hvg_res1, as.character(ann$final_cluster)),
  identical(sce$cluster_hvg_res0.8, as.character(ann$cluster_hvg_res0.8)))
rm(ann)
```

```
saveRDS(sce, file.path(paths$work_dir, "02_clustered.rds"))
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.5.2
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.
 ↪  framework/Versions/A/libBLAS.dylib
## LAPACK:
 ↪  /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
 ↪  LAPACK version 3.12.1
##
## locale:
## [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
```

```
##
## attached base packages:
## [1] stats4    stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
##  [1] igraph_2.2.1                scater_1.38.0
##  [3] ggplot2_4.0.1               scran_1.38.0
##  [5] Matrix_1.7-4                scuttle_1.20.0
##  [7] SingleCellExperiment_1.32.0 SummarizedExperiment_1.40.0
##  [9] Biobase_2.70.0              GenomicRanges_1.62.1
## [11] Seqinfo_1.0.0               IRanges_2.44.0
## [13] S4Vectors_0.48.0            BiocGenerics_0.56.0
## [15] generics_0.1.4              MatrixGenerics_1.22.0
## [17] matrixStats_1.5.0
##
## loaded via a namespace (and not attached):
##  [1] beeswarm_0.4.0      gtable_0.3.6        xfun_0.55
##  [4] ggrepel_0.9.6       lattice_0.22-7      vctrs_0.7.1
##  [7] tools_4.5.2         parallel_4.5.2      tibble_3.3.1
## [10] cluster_2.1.8.2     pkgconfig_2.0.3     BiocNeighbors_2.4.0
## [13] RColorBrewer_1.1-3  S7_0.2.1            dqrng_0.4.1
## [16] lifecycle_1.0.5     compiler_4.5.2      farver_2.1.2
## [19] statmod_1.5.1       tinytex_0.58        bluster_1.20.0
## [22] codetools_0.2-20    vipor_0.4.7         htmltools_0.5.9
## [25] yaml_2.3.12         pillar_1.11.1       BiocParallel_1.44.0
## [28] DelayedArray_0.36.0 limma_3.66.0        viridis_0.6.5
## [31] abind_1.4-8         tidyselect_1.2.1    rsvd_1.0.5
## [34] locfit_1.5-9.12     metapod_1.18.0      digest_0.6.39
## [37] BiocSingular_1.26.0 dplyr_1.2.0         fastmap_1.2.0
## [40] grid_4.5.2          cli_3.6.5           SparseArray_1.10.8
## [43] magrittr_2.0.4      S4Arrays_1.10.1     withr_3.0.2
## [46] edgeR_4.8.2         scales_1.4.0        ggbeeswarm_0.7.3
## [49] rmarkdown_2.30      XVector_0.50.0      otel_0.2.0
## [52] gridExtra_2.3       ScaledMatrix_1.18.0 beachmat_2.26.0
## [55] evaluate_1.0.5      knitr_1.51          viridisLite_0.4.2
## [58] irlba_2.3.7         rlang_1.1.7         Rcpp_1.1.1
## [61] glue_1.8.0          R6_2.6.1
```